

Understanding JDBC 2.0

Steve Close

Sun Certified Java Programmer

MTT

Intertech Inc.

Authorized Java Training Center

1020 Discovery Road, Suite 145

St. Paul, MN

www.intertech-inc.com



Intended Audience:

■ Anyone interested in Java!

- Developers
- Managers looking to understand the buzz
- Training Coordinators



Objectives:

- To walk away understanding JDBC API.
- To understand how JDBC achieves cross DBMS and cross platform database programming.
- To understand ways to develop Java distributed applications.
- Introduce new features added to JDBC in Java release 2.0.



Key Points of Presentation:

- Where did JDBC come from?
- Connecting and communicating with a relational database.
- Understanding the classes in the java.sql package.
- Understanding distributed java database applications.
- Quick look at new JDBC 2.0 features.



ODBC

- **ODBC** stands for Open Database Connectivity
 - Java borrows extensively this exceptionally successful standard.
 - If you understand ODBC your more then 1/2 way to understanding JDBC.
 - We begin with a little ODBC.



ODBC

- ODBC is widely accepted as a standard for developing code that is database independent.
- ODBC uses SQL to allow a program to talk to any DBMS that has an associated ODBC driver.



Structured Query Language

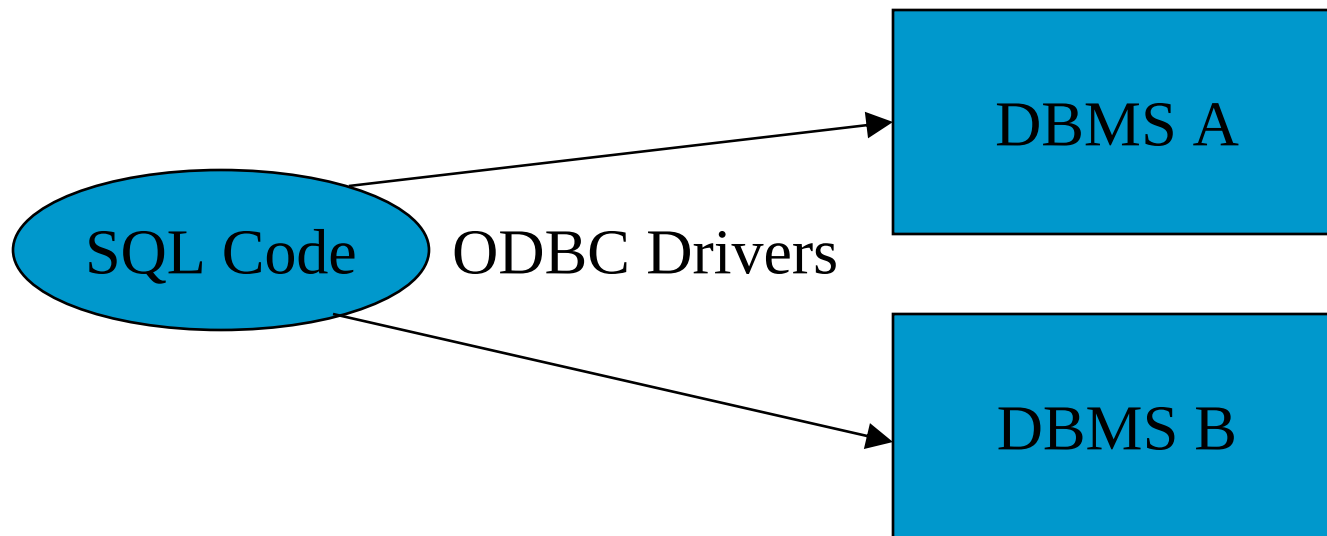
■ SQL: The language of relational databases.

- Create
- Delete
- Insert
- Select
- Update



ODBC Driver Concept

- Although the code to access your database never changes different databases can be 'swapped' by switching the drivers.





ODBC Benefits

- Database swapping allows for scalability in applications.
- Development is no longer dependant on proprietary database languages.
- Learn SQL while using PowerBuilder and you still have a viable skill when you become a Java programmer.



JDBC inspired by ODBC

- JDBC is not just ODBC, it was built from the ground up by Javasoft and many partners.
- JDBC borrow (steals) the SQL language so if you've used SQL your ½ way home.
- JDBC also has the database driver concept, but it needed to be modified slightly.
- Java is cross platform also, so JDBC programs are cross platform, and cross DBMS.



JDBC Driver types

- JDBC defines 4 driver types. Each is useful in different situations so be careful which you pick.
 1. JDBC-ODBC bridge plus ODBC driver.
 2. Native-API partly Java Driver.
 3. JDBC pure Java Driver.
 4. Native-protocol pure Java driver.



JDBC-ODBC bridge plus ODBC driver

- Translates JDBC calls into ODBC calls.
- Requires client to have ODBC driver installed so it's not useful for Applets.
- Used in two tier architectures.
- Used if the database is small, and doesn't support JDBC directly.



Native-API partly Java driver

- Translates JDBC into database specific protocol.
- Vendor supplied driver required.
- Not appropriate for Applets.



JDBC-Net pure Java Driver

- Communicates to a remote server which communicates to the database using ODBC, or database specific protocol.
- Pure Java implementation makes this ideas for applets.



Native-Protocol pure Java driver

- Use vendor specific database protocol.
- Highly vendor specific.
- Usually better performance because no translation to ODBC required.



Connecting to a database

- Irregardless of the driver communicating with the database follows the same steps!
 1. Load the driver.
 2. Connect to the database.
 3. Execute SQL Statements.



Loading Drivers

1. Using Class.forName().

```
String aDriver = "sun.jdbc.odbc.JdbcOdbcDriver";  
Class.forName(aDriver);
```

2. Loading the driver as a System Property.

```
Java -Djdbc.drivers=sun.jdbc.odbc.JdbcOdbcDriver  
YourProgramName
```



for Name?

- ForName is not a particularly descriptive method name. What actually happened?
 - ForName loads the driver class into your virtual machine.
 - ForName takes a driver class name String. See <http://java.sun.com/products/jdbc/jdbc.drivers.html> for information on current drivers.
 - The driver class automatically registers itself with the DriverManager (which we'll see soon)
 - The DriverManager can now create connections to appropriate databases.



Connecting to a Database

- Connecting to the Database is done by simply calling the static getConnection method from the DriverManager class.

```
Connection con = DriverManager.getConnection (  
    "jdbc:odbc:myDatabase",  
    "sclose",  
    "javaiscool");
```



OH CRUD!

- Now that we made our connection object using the DriverManager what to do with it?
 - Create
 - Read
 - Update
 - Delete



Statements

- In order to do any manipulation of the database we will need to use a statement object.
 - It's as simple as getting the connection was.

//Remember we called our Connection “con”

Statement stmt = con.**createStatement()**;

//Now we can use stmt



Creating tables

- To create a table use the executeUpdate method.

```
//Recall we called our Statement stmt  
stmt.executeUpdate("CREATE TABLE  
my_table(id INT, name CHAR(40))");
```



Creating Tables

- To create a new row in the table use the `executeUpdate` method.
 - `executeUpdate` returns the number of row effected.

```
//Recall we called our Statement stmt  
  
stmt.executeUpdate("INSERT INTO  
    my_table(id, name)  
    VALUES(1,'Steve')");
```



Creating Tables

- Assuming we insert a few more rows my_table might look like this.

Id	name
1	Steve
2	Horace
3	Drew
4	Gina



Reading Tables

- Use a 'select' statement to read a table.
 - Instead of executeUpdate use executeQuery

```
//recall Stmt?
```

```
ResultSet rs = stmt.executeQuery("SELECT id, name  
                                FROM my_table  
                                WHERE id=5");
```

```
//Hey! what's a ResultSet?
```



Reading Tables

- ResultSet are used to get information from the executed query.
- Key elements of ResultSet
 - next()
 - getXXX(by Index) //where XXX is the type
 - getXXX(by column name)
 - getMetaData()
 - close()



Reading Tables

//Quick result set read example

```
ResultSet rs = stmt.executeQuery("SELECT id, name  
    FROM my_table");
```

```
while(rs.next())
```

```
{
```

```
    System.out.print("ID: " + rs.getInt("id"));
```

```
    System.out.println("::Name: " + rs.getString(2));
```

```
}
```

```
rs.close();
```



Reading Tables

- Output from this read would look like the following.

```
ID:1::Name:Steve  
ID:2::Name:Horace  
ID:3::Name:Drew  
ID:4::Name:Gina
```



Updating Tables

- To update (yes you guessed it) use `executeUpdate` with an SQL update statement.
- We will see a better way when we get to Java release 2.

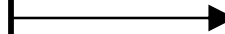
```
Stmt.executeUpdate("UPDATE my_table  
SET name='Steve Close'  
WHERE id=1 ");
```



Updating Tables

■ Resulting In:

Id	name
1	Steve
2	Horace
3	Drew
4	Gina



Id	name
1	Steve Close
2	Horace
3	Drew
4	Gina



Deleting

- Simply make a SQL Delete Statement

```
stmt.executeUpdate("DELETE FROM my_table  
WHERE id=1");
```



Deleting

■ Resulting in:

Id	name
1	Steve
2	Horace
3	Drew
4	Gina

Id	name
2	Horace
3	Drew
4	Gina



Prepared Statements

- To increase performance you can create prepared statements,
 - Precompiled by the Database before it is run.
 - Useful if a single Query will be run multiple times.
 - Create one using the Connection classes `prepareStatement` method.
 - Execute the prepared statement exactly like a regular Statement.



Prepared Statements

Example: Notice how PreparedStatement can accept parameterized statements.

```
PreparedStatement ps = con.prepareStatement("Select name  
                                         from my_table where id=?");  
  
ps.setInt(1,1);  
rs = ps.executeQuery();  
while (rs.next())  
    System.out.println(rs.getString("name"));
```



Stored Procedures

- Stored procedures are SQL programs stored in the Database that can be called from a client.
- They are faster and more efficient than prepared statements.
- They allow you to separate the concerns in an n-tier architecture.
- They can use two way parameters.
- On the down side they are vendor specific.



Stored Procedures

- Stored procedures are defined in the Database. Assuming sp_a_proc is the name of a stored procedure let's look at a very simple example.

```
CallableStatement cs;  
//con is an active connection  
cs = con.prepareCall("{call sp_a_proc}");  
cs.execute();
```



Stored Procedures

- Stored Procedures can accept parameters for input and output.
 - Input parameters are handled similar to prepared statements.
 - Input/output parameters require an extra registration step.



Stored Procedures

■ Example

```
CallableStatement cs = con.prepareCall(
    "{? = call sp_proc[(?,?)]}");
//First handle regular params
cs.setInt(2,13);
cs.setFloat(3,2.14);
//Next handle out params
cs.registerOutParameter(1,java.sql.Types.REAL);
cs.execute();
//retrieve the out parameter
float f = cs.getFloat(1);
```



Transactions

- What happens when two changes to one database depend on each other and one of them fails?
- Can I bundle Database changes or roll changes back when one fails?



Transactions

- Of course JDBC supports rollback.
- Simple steps to transaction management.
 - Disable 'Auto commit'
 - Execute multiple statements
 - Figure out if you should commit or rollback
 - Rollback, or commit.



Transactions

- Let's take a look at an example.

```
con.setAutoCommit(false);
try
{
    stmt.executeUpdate(someSql);
    stmt.executeUpdate(anotherSql);
    con.commit();
}
catch(Exception ex)
{
    con.rollback();
}
```



Metadata

- Metadata what does it mean?
 - Meta: more comprehensive, transcending
 - Data: information
- Transcendent Information? Actually it's information about the object that contains the information. In other words, detail about the database itself.



Metadata

■ How do I get the data?

//recall con is an active connection, and rs is a ResultSet

//for a connection

DatabaseMetaData dbmd = con.getMetaData();

//or for a resultset

ResultSetMetaData rsmd = rs.getMetaData();



Metadata

- ResultSetMetaData
 - Column Information
 - See API for more detail on info
- DatabaseMetaData
 - Product information
 - Driver Information
 - Supported feature info
 - Limitations



Halftime



- That was JDBC, but is that enough? It is the core so let's review for a minute (or 5).
 - Load Driver usually with `forName`.
 - Use `DriverManager` to get `Connection`.
 - Use `Connection` to get `Statement`.
 - Use `execute` method with SQL.
 - Explore `PreparedStatement`, `StoredProcedures`, `Metadata`, and `Transaction` further as you continue to use JDBC.



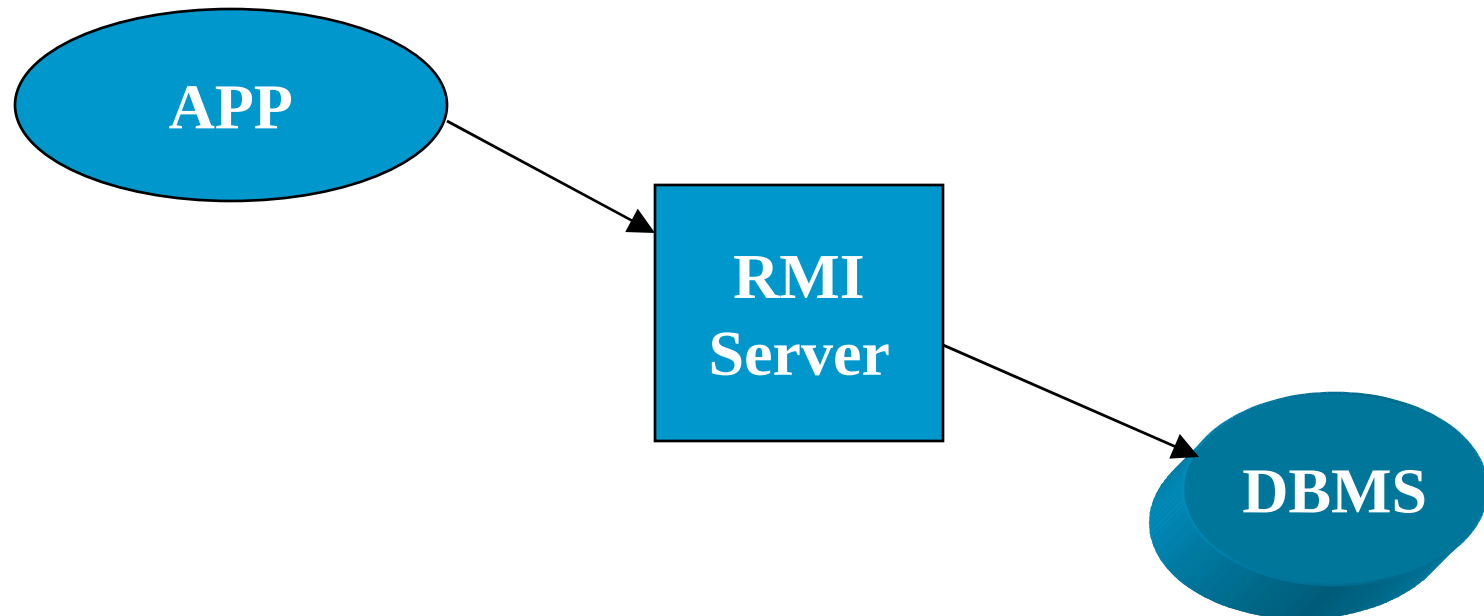
JDBC with RMI

- Three tier solutions are very common. Java provides a simple networking tool called RMI that we can use, so we will investigate how to use it. Why?
 - Remove some fear of Java terms.
 - Walk through the steps to implementing an RMI Server.
 - Should be fun.



JDBC with RMI

- The RMI Server isolates Database communication.





JDBC with RMI

- What exactly is RMI?
 - RMI stands for Remote Method Invocation.
 - Core to Java since JDK 1.1.
 - Way to define your object so objects in other VM's can access your methods.
 - Defines a simple way to pass objects across the network using serialization.
 - It's a definition built into Java so we can worry about sockets less often.



JDBC with RMI

- New Java buzz-tech words that you get for free with RMI.
 - Object Serialization: Fairly transparent way to have Object turn themselves into byte arrays.
 - JRMP: Java Remote Method Protocol is the way a server and it's client talk with one another.



JDBC with RMI

- Warning: There's more to object serialization and RMI than we see today.





JDBC with RMI

- Step by step RMI.
 1. Define remote interface.
 2. Develop remote object implementing your remote interface.
 3. Develop client.
 4. Compile.
 5. Generate stubs and skeletons.
 6. Start RMI registry.
 7. Start Remote Servers.
 8. Run Client.



JDBC with RMI

- Developing a remote interface.
 - Extend java.rmi.Remote
 - Throw java.rmi.RemoteException in all methods.

```
import java.rmi.*;
public interface PasswordCallInterface extends Remote
{
    public Access hasAccess(String name, String password)
        throws RemoteException;
}
```



JDBC with RMI

- The object returned by the remote interface must be serializable.

```
public class Access implements java.io.Serializable
{
    private boolean allowAccess;
    public Access(boolean axe)
    {
        allowAccess = axe;
    }
    public boolean allowAccess()
    {
        return allowAccess;
    }
}
```



JDBC with RMI

- Develop remote object implementing your remote interface.

```
import java.rmi.*;
import java.rmi.server.UnicastRemoteObject;

public class PasswordServer extends UnicastRemoteObject implements
                                PasswordCallInterface
{
    public PasswordServer() throws RemoteException
    {
        super();
    } //more to come
}
```



JDBC with RMI

- Create a main.
 - Creates an instance of the server.
 1. Invokes UnicastRemoteServer.
 2. Bind object to a port.
 3. Starts to listen on that port.
 - Instantiates some security manager.



JDBC with RMI

```
public static void main(String[] args)
{
    System.setSecurityManager(new RMISecurityManager());
    try
    {
        PasswordServer ps = new PasswordServer();
        Naming.bind("/PasswordServer",ps);
        System.out.println("Successfully bound");
    }
    catch(Exception ex)
    {
        System.out.println("Binding failed " + ex);
    }
}
```



JDBC with RMI

■ Implement the remote interface.

```
public Access hasAccess(String name, String password)
throws RemoteException
{
    // insert any database code here
    return new Access();
}
```



JDBC with RMI

■ Develop client.

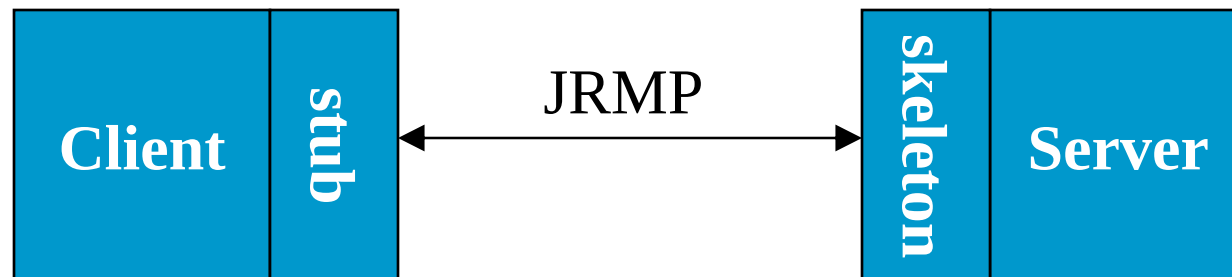
```
try
{
    PasswordCallInterface pci =(PasswordCallInterface)Naming.lookup
                                ("/PasswordServer");

    Access a = pci.hasAccess(s, s1);
    //do something with information
}
catch(Exception ex)
{
    System.out.println("naming failed " + ex);
}
```



JDBC with RMI

- Compile.
- Generate stubs and skeletons.
 - `rmic PasswordServer`





JDBC with RMI

- Start RMI registry.
 - start rmiregistry or rmiregistry&
- Start Remote Servers.
 - start java -Djava.rmi.server.codebase=http://...
-Djava.security.policy=c:\... PasswordServer
- Run Client.



JDBC with RMI

- Review of what's going on now.
 1. Client calls for remote object using Naming.lookup.
 2. RMI registry gives client reference to remote object.
 3. Client can now call all methods advertised in the remote interface.
 4. Server does all implantation work regardless of what virtual machine it's running on.
 5. Object that implement serializable can be passed (by value) back to the client.



JDBC with RMI

- Wrap up.
 - RMI is easy and very powerful.
 - We don't have enough time to fully explore RMI, but now that you've seen it, explore it!





What's new in JDBC 2.0?

- Remember you must find a driver that is JDBC 2.0 compatible to use these features.
 - Bi-directional cursors
 - Updateable Recordsets
 - Batch Updates
 - SQL 3 Datatypes
 - JDBC 2.0 Standard Extensions



What's new in JDBC 2.0?

- Forward and backward cursor scrolling.
 - Let's jump in and see the code.

```
Statement stmt =  
con.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,  
                    ResultSet.CONCUR_UPDATABLE);
```



What's new in JDBC 2.0?

- Types of Resultsets in 2.0
 1. TYPE_FORWARD_ONLY
 2. TYPE_SCROLL_SENSITIVE
 3. TYPE_SCROLL_INSENSITIVE
- Modifiers
 1. CONCUR_READ_ONLY
 2. CONCUR_UPDATABLE



What's new in JDBC 2.0?

- Moving backwards in a resultset.

```
ResultSet rs = stmt.executeQuery("Select ...");  
rs.afterLast();  
while(rs.previous())  
{  
    rs.getInt("id");  
    //...  
}
```



What's new in JDBC 2.0?

- Updating ResultSets without using `executeUpdate`.
 - If the Statement was created with the `CONCUR_UPDATABLE` constant you can make changes on the fly.

//Remember our old example?

```
rs = stmt.executeQuery("SELECT id, name FROM my_table");  
rs.first();  
rs.updateString("name", "Bill Bixby");
```



What's new in JDBC 2.0?

- New features of ResultSet.
 - updateXXX()
 - absolute(), relative(), beforeFirst(), afterLast(), first(), last()
 - isFirst(), isLast(), isBeforeFirst(), isAfterLast()
 - deleteRow()
 - insertRow()



What's new in JDBC 2.0?

- Batch Updates are simpler in JDBC 2.0.
 - Again it's easiest to see jumping right into the code

```
//recall con is an active connection
con.setAutoCommit(false);
Statement stmt = con.createStatement();
stmt.addBatch("INSERT INTO ...");
stmt.addBatch("INSERT INTO ...");
stmt.addBatch("INSERT INTO ...");
stmt.executeBatch();
con.commit(); //commit can throw a BatchUpdateException
```



What's new in JDBC 2.0?

- Support for SQL3 Datatypes each of the following can be found as objects in java.sql, support for setting, getting and updating has been added to ResultSet.
 - Struct
 - Ref
 - Blob
 - Clob
 - Array



What's new in JDBC 2.0?

- JDBC Standard extensions are located in javax.sql.
 1. Rowsets
 2. DataSource interface
 3. Connection Pooling
 4. Distributed Transaction Support



What's new in JDBC 2.0?

■ Rowsets

- Container for a set of rows.
- Used to transport data across network.
- Extends and contains all functionality of ResultSet.
- Can update row, or disconnect from db to minimize size.
- Built in listeners so tie cursor scrolling to a GUI object like a table.



What's new in JDBC 2.0?

- DataSource works with Java Naming and Directory Interface (JNDI) to create a better way to connect to a data source.
 - Alternative to DriverManager
 - More Portable
 - Easier to maintain



What's new in JDBC 2.0?

- The DataSource class is implemented with Connection Pooling. Connection Pooling automatically pools connections when they are opened. Creating new connections is costly so this can dramatically improve performance by reducing the number of connections that are made.



What's new in JDBC 2.0?

- Distributed transactions is a way to include multiple database servers in a single transaction. It is transparent to the user except that commit, rollback should never be manually called. The middle tier contains several objects that handle distributed transactions fairly invisible to the user.



Presentation Summary!

- JDBC built in a similar way to ODBC enables us to write DBMS independent code. Java provides cross platform support so together we have cross platform cross DBMS code!
- JDBC is contained in the java.sql package and provides simple to use objects to access any Database provided you have a driver.

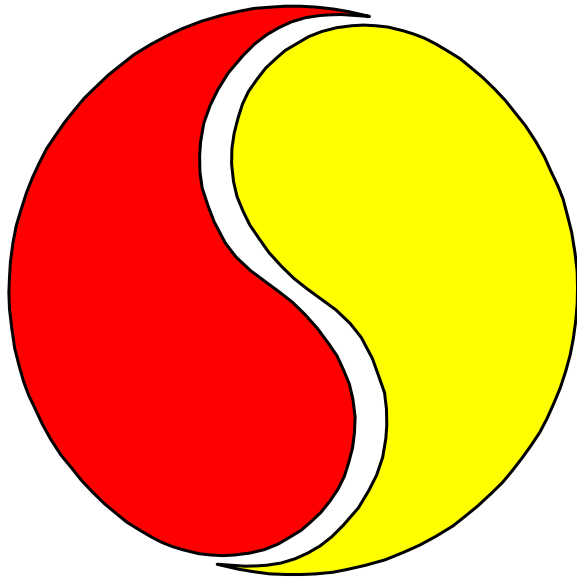


Presentation Summary!

- RMI can be use to distribute your database access across multiple Virtual machines
- With the release of 2.0 JDBC has grown to be more flexible and more powerful.



Presentation Complete!



Please see your handouts,
or www.intertech-inc.com
for a break down of
Intertech's Java TM course
offerings including

*“Java TM Programming
Language SL276”*

*“Beyond CGI:Developing
JavaTM Servlets SL-311“*

Thanks for attending! (Do we have time for questions???)